Studio Nx

P1-18 Phase 1 완료 기준 시험 — 묶음 설명 합본 (백엔드 93~95)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-18 - 완료 기준 시험/93_acceptance_bundle_a.md
2. ai-notes/backend(fastAPI_Python)/P1-18 - 완료 기준 시험/94_acceptance_bundle_b.md
3. ai-notes/backend(fastAPI_Python)/P1-18 - 완료 기준 시험/95_acceptance_bundle_c.md

Backend(FastAPI/Python) 93 — 묶음 A: 가입, 로그인, 마이페이지, 비밀번호, 중복 가입

항목	내용
파일	`studionx/backend/tests/test_phase1_acceptance.py` (묶음 A, 시험 6개)
작업	P1-18 완료 기준 시험: PRD 01 §10의 1, 2, 3번 완료 기준을 실제 요청으로 확인
작성	Claude (AI), 2026-10-13, 사용자 검토 (묶음 단위)
이전 · 다음	(P1-18 첫 묶음) · 94 묶음 B (다음)

1. 하는 일

작업마다 만든 시험이 가입, 로그인, 대시보드를 **따로따로** 확인하고 있었습니다. 이 묶음은 그것들이 **하나의 흐름으로 이어지는지**와, 기존 시험에 빠져 있던 두 곳(닉네임 중복의 요청 시험, DB 전체의 평문 검사)을 채웁니다. 모든 시험은 실제 앱과 실제 MySQL로 돌고, 시험이 끝나면 되돌려집니다.

시험	완료 기준	확인하는 것
`test_a1_signup_login_my_page_one_flow`	1	가입(201) → 로그인(200) → `/users/me` → `/me/dashboard`를 차례로 부릅니다. 프로젝트를 만들면 대시보드의 최근 프로젝트와 이어 하기에 나타나고, 로그인 때 받은 쿠키로 `/auth/refresh`가 새 Access Token을 줍니다
`test_a2_dashboard_and_me_need_login`	4의 요청 쪽	`/users/me`, `/me/dashboard`, `/projects`가 토큰 없이 401 `UNAUTHORIZED`입니다 (화면의 로그인 모달 이동은 화면 시험 범위)
`test_a3_password_is_never_stored_in_plain_text`	3	해시가 bcrypt 모양(60자, `$2`로 시작)이고 올바른 비밀번호만 통과합니다. 사용자 행의 **모든 칸**에 평문이 없고, 같은 비밀번호도 해시가 다르며(소금), 가입과 로그인 응답에 비밀번호와 해시가 없습니다
`test_a4_duplicate_email_and_nickname_cannot_sign_up`	2	이메일이 같으면 409 `EMAIL_TAKEN`, 닉네임이 같으면 409 `NICKNAME_TAKEN`입니다. 대소문자만 달라도 막히고, 거절된 가입은 행을 남기지 않습니다
`test_a5_signup_creates_no_project_member_rows`	9의 앞부분	가입만으로는 프로젝트 멤버 행이 생기지 않습니다 (OWNER 행은 프로젝트를 만들 때 생깁니다)
`test_a6_login_keeps_signup_account_and_updates_last_login`	1	가입 직후 마지막 로그인은 비어 있고, 로그인하면 UTC 현재 시각으로 기록됩니다

2. 코드 (뼈대 부분)

```
@pytest.fixture
def client(db):
    app.dependency_overrides[get_db] = lambda: db   # 앱이 시험용 세션을 쓰게 함
    yield TestClient(app)
    app.dependency_overrides.clear()
```

```
def sign_up(client, name, **fields):
    return client.post(f"{API}/auth/signup", json=signup_body(name, **fields))

def log_in(client, name, password=PASSWORD, remember=False): ...
def bearer(res):
    return {"Authorization": f"Bearer {res.json()['access_token']}"}
```

`signup_body(name)`은 이메일 `{name}@example.com`, 닉네임 `{name}_nick`을 만듭니다. 시험마다 다른 `name`을 써서 서로 겹치지 않게 합니다.

평문 검사 부분 (A3)

```
for column in inspect(User).columns:
    value = getattr(user, column.key)
    assert PASSWORD not in str(value), column.key
```

식	뜻
`inspect(User).columns`	`users` 테이블의 모든 칸 목록입니다
`getattr(user, column.key)`	그 칸의 값을 꺼냅니다
`PASSWORD not in str(value)`	어느 칸이든 평문이 들어 있으면 실패합니다. 해시 칸만이 아니라 모든 칸을 보므로, 나중에 로그 칸이나 메모 칸이 생겨도 같은 시험이 지켜 줍니다

3. 왜 이렇게

선택	이유
서비스를 직접 부르지 않고 **HTTP 요청**으로	화면이 실제로 쓰는 경로(라우터, 의존성, 오류 처리기)를 모두 지나갑니다
한 시험에 흐름을 이어서	완료 기준이 "가입 → 로그인 → 마이페이지"라는 **연결**이어서, 각 단계가 맞아도 연결이 끊기면 이 시험만 잡습니다
모든 칸을 검사	해시 칸만 보면 다른 칸에 평문이 새는 경우를 놓칩니다
닉네임 중복을 요청으로	서비스 시험은 있었지만 라우터가 409를 올바른 JSON으로 내보내는지는 이 묶음이 처음 확인합니다
거절된 가입의 행 수 확인	가입이 실패했는데 일부 행이 남는 일(트랜잭션 누수)을 막습니다

4. 결과

6개 모두 처음 돌렸을 때 통과했습니다. 빠진 곳을 메우는 시험이어서, 기존 코드가 이미 완료 기준 1, 2, 3번을 만족한다는 확인이며 코드 수정은 없었습니다.

Backend(FastAPI/Python) 94 — 묶음 B: 다른 회원의 프로젝트 차단

항목	내용
파일	`studionx/backend/tests/test_phase1_acceptance.py` (묶음 B, 시험 9개)
작업	P1-18 완료 기준 시험: PRD 01 §10의 10번 "다른 회원의 프로젝트는 API로도 조회, 수정, 삭제할 수 없다"
작성	Claude (AI), 2026-10-13, 사용자 검토 (묶음 단위)
이전 · 다음	93 묶음 A · 95 묶음 C (다음)

1. 하는 일

기존 시험은 프로젝트의 **멤버**(편집자, 뷰어)가 무엇을 할 수 있는지를 확인했습니다. 이 묶음은 프로젝트와 **아무 관계가 없는 회원**이 실제로 로그인한 상태에서 무엇을 할 수 있는지를 확인합니다. 소유자와 관계없는 회원은 가입과 로그인을 거쳐 만든 두 계정입니다.

시험	확인하는 것
`test_b1`	관계없는 회원의 조회, 수정, 삭제가 모두 404 `NOT_FOUND`("Project not found.")입니다. 응답에 제목과 설명이 새지 않고, 소유자의 프로젝트는 그대로(제목, 설명, 목록 1건)입니다
`test_b2`	남의 프로젝트와 **없는 번호**의 응답(상태와 본문)이 조회, 수정, 삭제 모두 같습니다. 번호를 바꿔 가며 존재 여부를 알아낼 수 없습니다
`test_b3`	잘못된 본문(`title: null`)으로 수정해도 남의 프로젝트와 없는 번호가 같은 422입니다
`test_b4`	관계없는 회원의 목록, 검색(제목을 정확히 알아도), 대시보드의 최근과 이어 하기 어디에도 소유자의 프로젝트가 없습니다. 소유자에게는 그대로 보입니다
`test_b5`	두 회원이 각자 프로젝트를 만들면 목록은 각자의 것뿐이고, 서로의 번호를 알아도 열리지 않습니다
`test_b6`	토큰이 없거나 잘못된 토큰이면 404가 아니라 401입니다 (조회, 수정, 삭제, 목록 모두)
`test_b7`	리프레시 쿠키 값을 Access Token처럼 써도 프로젝트 API가 열리지 않습니다
`test_b8`	멤버 행을 넣으면 보이고(뷰어는 수정 403), 멤버 행을 지우면 다시 404로 닫힙니다
`test_b9`	소유자가 삭제하면 소유자와 관계없는 회원 모두에게 404이고, 소유자의 대시보드에서도 사라집니다

2. 코드 (뼈대 부분)

```
@pytest.fixture
def two_users(client):
    owner = member_headers(client, "acc_owner")
    stranger = member_headers(client, "acc_stranger")
    created = client.post(PROJECTS, json={"title": "Secret Song", "description": "private"},
                          headers=owner)
    return owner, stranger, created.json()["project_id"]


def assert_project_not_found(res):
    assert res.status_code == 404
    error = res.json()["error"]
```

```
    assert (error["code"], error["message"]) == ("NOT_FOUND", "Project not found.")
    assert "Secret Song" not in res.text and "private" not in res.text
```

B2의 핵심 (같은 응답)

```
foreign = send(f"{PROJECTS}/{pid}")
missing = send(f"{PROJECTS}/999999999")
assert (foreign.status_code, foreign.json()) == (missing.status_code, missing.json())
```

식	뜻
`send(url)`	조회, 수정, 삭제 세 가지를 같은 방식으로 불러 보는 도우미입니다
`foreign`, `missing`	남의 프로젝트에 대한 응답과 없는 번호에 대한 응답입니다
`(상태, 본문) == (상태, 본문)`	둘이 **완전히** 같아야 통과입니다. 문구 한 글자라도 다르면 존재 여부가 새는 것이므로 실패합니다

3. 왜 이렇게

선택	이유
403이 아니라 **404**	403은 "있지만 못 본다"를 알려 줍니다. 사용자 결정(2026-10-08 A안)대로 남의 프로젝트가 있는지조차 알려 주지 않습니다
없는 번호와 응답을 비교	상태 코드만 같아도 본문이나 헤더가 다르면 구분할 수 있습니다. 본문까지 비교합니다
응답에 제목과 설명이 없는지 확인	오류 응답에 내용이 섞이는 실수를 막습니다
잘못된 본문(422)도 확인	FastAPI는 본문 검사를 먼저 하므로, 존재 여부와 관계없이 같은 응답이어야 합니다
토큰 오류는 401	"로그인이 필요함"이지 "권한 없음"이 아닙니다. 404와 섞이면 화면이 잘못된 안내를 합니다
리프레시 쿠키를 Access Token으로 써 보기	두 토큰은 서명 방식이 같아서 구분하지 않으면 재발급 전용 토큰으로 API를 쓸 수 있습니다 (P1-03의 `typ` 구분이 막아 줍니다)
멤버 행을 DB에서 직접 넣고 지움	멤버 초대 기능은 Phase 6이어서 API가 없습니다. 접근 판단이 **멤버 행**에 달려 있다는 것을 직접 확인합니다

4. 시험이 의미 있는지 확인 (일부러 망가뜨리기)

처음부터 통과한 시험이 정말 문제를 잡아내는지 확인하려고, `get_membership`을 "내 멤버 행이 없으면 그 프로젝트의 **아무 멤버나** 인정"하도록 임시로 바꿨습니다.

결과	내용
실패한 시험	B1, B2, B5, B8 (4개)
통과한 시험	나머지 11개 (이 코드가 닿지 않는 시험들)
되돌린 뒤	15개 모두 통과, `app/` 아래에 바뀐 파일 없음

5. 결과

9개 모두 통과했습니다. 코드 수정은 없었고, 기존 코드가 완료 기준 10번을 이미 만족한다는 확인입니다.

Backend(FastAPI/Python) 95 — 묶음 C: 탈퇴한 계정

항목	내용
파일	`studionx/backend/tests/test_phase1_acceptance.py` (묶음 C, 시험 8개)
작업	P1-18 완료 기준 시험: PRD 01 §10의 7번 "탈퇴한 계정은 로그인되지 않는다" (FR-108 재가입 규칙 포함)
작성	Claude (AI), 2026-10-13, 사용자 검토 (묶음 단위)
이전 · 다음	94 묶음 B · (P1-18 끝, 개요는 61번)

1. 하는 일

가입, 로그인, 프로젝트 하나를 만든 회원이 **실제 요청으로 탈퇴**하는 상황을 만들고, 그 뒤의 모든 문이 닫혔는지와 3일 재가입 규칙이 지켜지는지를 확인합니다. 시험 시작은 `withdrawn` 도우미가 맡습니다 (회원 번호, 탈퇴 전에 받은 헤더, 프로젝트 번호를 돌려줍니다).

시험	확인하는 것
`test_c1`	탈퇴 전에 받아 둔 Access Token으로 `/users/me`, `/me/dashboard`, `/projects`, `/projects/{id}`, 프로젝트 생성, 정보 수정을 모두 시도하면 401입니다. 재발급도 401입니다
`test_c2`	탈퇴한 계정으로 로그인하면 **없는 이메일과 응답이 완전히 같습니다**(401 `INVALID_CREDENTIALS`). 쿠키는 받지 않습니다
`test_c3`	비밀번호가 틀리면 400 `WRONG_PASSWORD`이고 아무것도 바뀌지 않습니다. 계속 로그인 상태이고 다시 로그인도 됩니다
`test_c4`	탈퇴 직후에도 이메일과 닉네임은 바뀌지 않고, `status`는 `DELETED`, `deleted_at`이 채워집니다
`test_c5`	3일 안에는 중복 확인 API가 `available: false`, `reason: REJOIN_TOO_SOON`, 가능해지는 시각을 줍니다. 같은 이메일로 가입하면 409 `REJOIN_TOO_SOON`(세부 정보에 시각 포함), 같은 닉네임으로 가입해도 409 `REJOIN_TOO_SOON`입니다
`test_c6`	3일 경계가 정확합니다. 탈퇴한 지 2일 23시간 59분이면 409, 3일 1분이면 가입됩니다 (201)
`test_c7`	3일 뒤에는 같은 이메일과 닉네임으로 다시 가입됩니다. 새 계정은 새 번호이고, 예전 행은 `DELETED`로 남고 이메일은 `deleted+번호@withdrawn.invalid`, 닉네임은 `deleted_user_…`로 비워집니다. 새 계정은 예전 프로젝트를 물려받지 않고(목록 비어 있음, 예전 프로젝트 404), 예전 비밀번호로는 새 계정에 못 들어갑니다. 중복 확인(GET)은 DB를 바꾸지 않습니다
`test_c8`	탈퇴한 소유자의 프로젝트는 다른 멤버(편집자)가 계속 읽고 고칠 수 있습니다. 응답에 탈퇴한 소유자의 이메일, 닉네임이 보이지 않습니다

2. 코드 (뼈대 부분)

```
@pytest.fixture
def withdrawn(client, db):
    headers = member_headers(client, "acc_gone")
    user_id = client.get(f"{API}/users/me", headers=headers).json()["user_id"]
    pid = client.post(PROJECTS, json={"title": "Left Behind"}, headers=headers).json()["project_id"]
    assert withdraw(client, headers).status_code == 204
    return user_id, headers, pid


def set_deleted_ago(db, user_id, **ago):
```

```
    user = db.get(User, user_id)
    user.deleted_at = datetime.now(UTC).replace(tzinfo=None) - timedelta(**ago)
    db.flush()
    db.expire_all()
```

식	뜻
`withdraw(client, headers, password)`	`DELETE /users/me`에 비밀번호를 본문으로 보냅니다 (`client.request`로 `DELETE`에 본문을 실었습니다)
`set_deleted_ago(db, id, days=3, minutes=1)`	탈퇴 시각을 과거로 돌려서 "3일이 지난" 상황을 만듭니다. 시험에서 실제로 3일을 기다릴 수 없기 때문입니다
`db.expire_all()`	세션이 기억한 값을 버려서, 서버가 바꾼 값을 다시 읽게 합니다
`timedelta(**ago)`	`days=2, hours=23, minutes=59`처럼 넘긴 값을 기간으로 만듭니다

3. 왜 이렇게

선택	이유
탈퇴 **전에** 받은 Access Token을 씀	Access Token은 만료(30분)까지 서명이 유효합니다. 서버가 매 요청마다 계정 상태를 확인해야 막히므로, 이 시험이 그 확인이 살아 있는지 봅니다
로그인 응답을 없는 이메일과 비교	"탈퇴한 계정이다"를 알려 주면 누가 탈퇴했는지 알아낼 수 있습니다 (FR-103, 108)
3일 경계를 1분 단위로	경계 오류(`<`와 `<=`, 시간대 혼동)는 정확히 경계에서만 드러납니다
중복 확인 API는 DB를 바꾸지 않음을 확인	읽기 요청이 계정 이름을 비우면 누구나 남의 탈퇴 계정을 풀 수 있게 됩니다. 가입할 때만 비웁니다
새 계정이 프로젝트를 물려받지 않는지	이메일이 같아도 번호가 다른 별개의 회원입니다
탈퇴한 소유자의 프로젝트를 남김	사용자 결정(2026-10-03)입니다. 다른 멤버의 작업이 사라지지 않습니다

4. 시험이 의미 있는지 확인 (일부러 망가뜨리기)

확인한 뒤 모두 되돌렸고, `app/` 아래에는 바뀐 파일이 없습니다.

바꾼 코드	실패한 시험
재가입 대기 `REJOIN_WAIT_DAYS`를 3일에서 1일로	C6 (경계)
토큰 검사에서 "정상 계정만 허용"을 끔 (`deps.py`)	C1 (탈퇴 뒤 잠금)

5. 결과

8개 모두 통과했습니다. 코드 수정은 없었고, 기존 코드가 탈퇴 관련 완료 기준을 이미 만족한다는 확인입니다. 정책이 불분명해서 질문한 곳은 없었습니다.